8. Integración y Testing

Este capítulo documenta cómo se verificó el sistema y qué resultados se obtuvieron. Retoma la estrategia de la pirámide de pruebas definida en el plan de pruebas de la Tesis (unitarias, de integración y de extremo a extremo) y la aplica a lo que existe hoy construido y ejecutable: la aplicación web progresiva (PWA) de demostración y el módulo de ingesta IoT (MQTT → PostgreSQL). Los resultados que figuran como «Aprobada» fueron obtenidos ejecutando las pruebas; lo que todavía no se ejecutó figura como «Pendiente» y no se presenta como logrado.

8.1 Alcance y entorno de prueba

Elemento	Detalle
Componentes bajo prueba	PWA de demostración (frontend, HTML/JS) · Módulo IoT: esquema PostgreSQL, ingesta MQTT, motor de alertas, simulador de sensores · Trigger de auditoría
Fuera de esta etapa	Backend Django completo (clientes, facturación AFIP, reportes) y pruebas con hardware real en el galpón (ver 8.7)
Base de datos	PostgreSQL 16.15 (compatible con la versión 15 especificada)
Mensajería	Cliente paho-mqtt 2.1.0 · broker embebido amqtt 0.12.1 para las pruebas automáticas (en producción: Eclipse Mosquitto 2.x)
Herramientas	pytest 9.2 (pruebas del módulo IoT) · Chromium automatizado con Playwright, pantalla móvil 390 × 844 px (pruebas de interfaz)
Repositorio	github.com/Mcarnielli/Ingenieria-en-SFT-Tilcor — carpetas backend/iot, backend/db y frontend

8.2 Estrategia de integración

La integración se hizo de forma incremental, de abajo hacia arriba, de modo que cada capa se probó antes de conectarla con la siguiente:

1. Reglas de negocio aisladas: función de evaluación de umbrales por variable (sin base de datos ni red).
2. Reglas + base de datos: procesamiento de una lectura (validación, guardado, alerta) contra PostgreSQL real.
3. Reglas + base + mensajería: el simulador publica por MQTT, el broker entrega, la ingesta guarda y alerta (flujo de extremo a extremo).
4. Interfaz: la PWA se recorre como lo haría cada perfil de usuario.
5. Hardware real: el mismo flujo con un sensor físico en el galpón (pendiente de ejecución en TILCOR).

Para poder probar sin depender del hardware se construyó un simulador de sensores que publica en los mismos topics y con el mismo formato JSON que usarán los equipos reales (por ejemplo, gallinero/1/sensores/humedad/1 con {"sensor_id":3,"valor":62.4,"unidad":"%"}). El simulador puede forzar una anomalía para provocar una alerta.

8.3 Pruebas unitarias y de integración del módulo IoT

Se ejecutaron 29 pruebas automáticas; las 29 resultaron aprobadas (tiempo total ≈ 7 s).

ID	Caso de prueba	Tipo	Resultado esperado	Estado
IOT-01 a 17	Umbrales por variable (temperatura, humedad, calidad de aire, temperatura de agua, flujo): 17 valores en los límites ok / warning / crítico	Unitaria	Cada valor se clasifica en el nivel correcto según la tabla de umbrales (Tesis 10.4)	Aprobada
IOT-18	Lectura válida de temperatura (25,6 °C)	Integración (DB)	Se guarda 1 lectura en estado ok, sin alerta	Aprobada
IOT-19	Tres lecturas críticas seguidas de humedad (38, 37, 36 %), luego normalización y nueva anomalía	Integración (DB)	Una sola alerta por episodio; al normalizarse y volver a fallar se genera la segunda (2 en total)	Aprobada
IOT-20 a 27	Lecturas inválidas: JSON roto, valor no numérico, falta el campo valor, valor fuera de rango físico (500 °C; −5 % humedad), tipo de sensor desconocido, sensor no registrado, topic inválido	Integración (DB)	Se descartan, no se guardan y se registran en el log (8 casos)	Aprobada
IOT-28	Flujo completo: el simulador publica los 7 sensores del galpón 1 y luego una anomalía de humedad	Extremo a extremo (MQTT)	7 lecturas guardadas sin alertas; tras la anomalía, 1 alerta crítica	Aprobada
IOT-29	Latencia entre publicar una lectura y verla guardada	Rendimiento	< 2 s	Aprobada

Tabla 8.1 — Resultados de las pruebas automáticas del módulo IoT.

La latencia medida fue de 11 ms, pero se obtuvo con broker y base en la misma máquina, por lo que solo demuestra que el procesamiento propio es despreciable; la latencia real dependerá de la red del galpón y se medirá en la prueba con hardware (HW-02).

Además se ejecutaron los scripts reales por línea de comandos (ingest.py y simulador.py) contra un broker MQTT: 5 ciclos de 7 sensores dejaron 35 lecturas guardadas y 2 alertas críticas (humedad 1 y humedad 2, 38 %), como se esperaba.

8.4 Pruebas de la interfaz (demo funcional)

Se recorrieron los flujos principales de la PWA con un navegador automatizado, tomando capturas en cada paso (reproducibles con el script del repositorio). El conjunto completo de capturas está en el Manual de Usuario, que se entrega como documento aparte para el cliente.

ID	Caso de prueba	Resultado obtenido	Estado
UI-01	Ingreso con Admin / contraseña correcta	Muestra «Bienvenido, Administrador» y el panel de inicio con KPIs	Aprobada
UI-02	Ingreso con contraseña incorrecta	Mensaje «Usuario o contraseña incorrectos»; no se accede	Aprobada
UI-03	Registro diario: 2.940 huevos, 3.520 aves, 2 muertas	Calcula 83,52 % de postura (verde) y confirma «Registro guardado correctamente»; aparece en el historial	Aprobada
UI-04	Registro diario con 2.500 huevos (postura 71,02 %, más de 5 pp bajo tabla)	Postura en rojo y aviso «Alerta de producción» con Confirmar / Cancelar al supervisor	Aprobada (tras corregir DEF-01)
UI-05	Perfil Empleado (Usuario)	No muestra la pestaña ni la sección Ventas; resto de módulos disponible	Aprobada
UI-06	Panel IoT	Muestra los 6 sensores con estado (humedad 1 en «Atención» a 44,2 %) y gráficos de 6 h	Aprobada

ID	Caso de prueba	Resultado obtenido	Estado
UI-07	Sanidad	Muestra viabilidad por línea, alerta de mortalidad semanal y protocolo de alertas	Aprobada

Tabla 8.2 — Resultados de las pruebas de interfaz sobre la demo desplegada en Vercel.

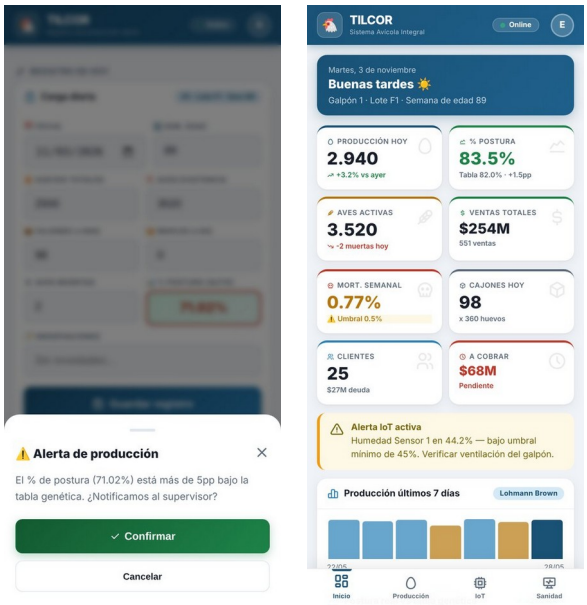


Figura 8.1 — UI-04: aviso de alerta de producción (izq.) · UI-05: perfil Empleado sin acceso a Ventas (der.).

8.5 Defectos detectados y corregidos

ID	Descripción	Severidad	Corrección	Estado
DEF-01	La alerta de postura baja nunca se mostraba: una función openModal declarada dos veces provocaba recursión infinita («Maximum call stack size exceeded») y ningún cuadro de diálogo podía abrirse	Alta	Se unificó en una sola función. Se repitió UI-04 con resultado correcto (commit 21e1980)	Corregido
DEF-02	El Manual de Usuario indicaba una dirección web inexistente	Baja	Reemplazada por https://tilcorvercel.vercel.app	Corregido
DEF-03	El índice del informe tenía números de página que no coincidían con el documento	Baja	Índice corregido	Corregido

Tabla 8.3 — Registro de defectos.

DEF-01 es un ejemplo del valor de probar el flujo completo: la lógica de la alerta era correcta, pero un error de integración entre dos funciones la volvía inalcanzable, algo que una lectura del código no mostraba a simple vista.

8.6 Pruebas de aceptación con el cliente

La aceptación se hará por fase, tal como establece el Contrato de Alcance: al cierre de cada fase se realiza una reunión de revisión con TILCOR y se verifican los criterios de aceptación del módulo contra el sistema funcionando. El pago del 70 % del precio de la fase queda sujeto a esa aceptación.

8.7 Pruebas con hardware real en TILCOR

Pendientes de ejecución durante la visita a la empresa. La tabla se completa con los resultados obtenidos en el lugar.

ID	Caso de prueba	Criterio de aprobación	Resultado	Estado
HW-01	El nodo (ESP32 + sensor de temperatura y humedad) se conecta al Wi-Fi del galpón y publica en MQTT	Las lecturas llegan al broker y se guardan en la base		Pendiente
HW-02	Latencia del dato real hasta la base, con la red del galpón	< 2 s		Pendiente
HW-03	Comparación con un termómetro de referencia, 15 a 30 min	Diferencia ≤ 1 °C		Pendiente
HW-04	Corte y vuelta del Wi-Fi	El nodo se reconecta solo y reanuda la publicación		Pendiente
HW-05	Valor anómalo forzado (por ejemplo, calentar o humedecer el sensor)	Se genera la alerta correspondiente		Pendiente

Tabla 8.4 — Pruebas con hardware real (a completar).

8.8 Limitaciones y trabajo futuro

- No se midió el porcentaje de cobertura de código; el objetivo definido en el plan de pruebas (80 % en producción y clientes) corresponde a la etapa del backend Django.
- No se realizaron pruebas de carga ni de seguridad (penetración); se prevén en la Fase 5.
- Las pruebas automáticas usan un broker embebido en lugar de Mosquitto; el protocolo es el mismo (MQTT 3.1.1), pero la prueba con Mosquitto y TLS se hará en la puesta en marcha.
- La demo guarda los datos en memoria del navegador y no tiene backend; los datos persistentes y el login con JWT se implementan en el backend.

9. Especificaciones del Sistema

Este capítulo define dónde y cómo se instala el sistema, con qué recursos de hardware y software, y con qué medidas de seguridad y auditoría. Distingue lo que ya está en funcionamiento (la demo en Vercel y el módulo IoT probado) de lo que corresponde a la puesta en producción.

9.1 Despliegue

9.1.1 Arquitectura de despliegue

Nodo	Componentes	Estado actual
Dispositivos del usuario	PWA instalable en celular, tablet o PC (navegador Chrome/Edge). Modo sin conexión con Service Worker	En funcionamiento (demo)
Hosting del frontend	Vercel (CDN con HTTPS). Despliegue automático desde la rama principal del repositorio	En funcionamiento: tilcorvercel.vercel.app
Servidor de aplicación (VPS Linux)	Nginx (proxy inverso + TLS) → Django/Gunicorn (API REST) · Django Channels (WebSocket) · Celery + Redis (tareas y reportes) · PostgreSQL con TimescaleDB · Mosquitto (broker MQTT)	Módulo IoT y base probados en local; el resto, a instalar en Fases 2 a 5
Nodos de campo (galpón)	ESP32 con sensores de temperatura, humedad, calidad de aire y flujo de agua; publican por MQTT sobre el Wi-Fi del galpón	Simulados; prueba real pendiente (cap. 8.7)
Servicios externos	Don Gestión (iframe/REST) y AFIP-WSFE (SOAP)	Don Gestión integrado por iframe; AFIP en Fase 4

Tabla 9.1 — Nodos del despliegue.

9.1.2 Contenedores y puertos

En el servidor, cada servicio corre en un contenedor Docker orquestado con Docker Compose, lo que permite reproducir la instalación y actualizarla de forma controlada. El archivo docker-compose.yml completo y las variables de entorno están en la Tesis (capítulo 15); la única diferencia en producción es que el broker Mosquitto no se publica en el puerto 1883 sin cifrado, sino únicamente en el 8883 con TLS.

Servicio	Puerto	Exposición	Función
Nginx	443 (80 redirige)	Pública (HTTPS)	Entrada única, certificado TLS
Mosquitto	8883	Pública (MQTT sobre TLS, con usuario y clave)	Recibe lecturas de sensores
Django / Gunicorn	8000	Interna	API REST y WebSocket
PostgreSQL + TimescaleDB	5432	Interna (nunca expuesta)	Datos del sistema y series de tiempo
Redis	6379	Interna	Cola de Celery y canales WebSocket

Tabla 9.2 — Servicios, puertos y exposición.

9.1.3 Procedimiento de despliegue

1. Contratar el VPS (Ubuntu Server 24.04 LTS) y apuntar un dominio al servidor.
2. Instalar Docker y Docker Compose; clonar el repositorio.

3. Cargar las variables de entorno (claves, base de datos, credenciales MQTT y de AFIP) en un archivo .env fuera del repositorio.
4. Aplicar el esquema de base de datos (schema.sql, auditoria.sql y migraciones del backend) y activar TimescaleDB sobre la tabla de lecturas.
5. Emitir el certificado TLS (Let's Encrypt) y levantar los servicios con Docker Compose.
6. Crear los usuarios y roles iniciales y registrar los sensores (tabla sensores).
7. Configurar cada ESP32 con la dirección del broker y sus credenciales, y verificar que las lecturas lleguen.
8. Programar los respaldos (9.4) y ejecutar la prueba de aceptación de la fase.

El frontend se publica en Vercel con cada cambio en el repositorio, sin intervención manual. El procedimiento de despliegue en el VPS es el previsto y se ejecutará en la puesta en marcha; no fue ejecutado en esta instancia.

9.2 Hardware mínimo y óptimo

9.2.1 Servidor

Recurso	Mínimo	Óptimo
Procesador	2 vCPU	4 vCPU
Memoria RAM	4 GB	8 GB
Almacenamiento	40 GB SSD	80 GB SSD (NVMe)
Red	100 Mbps, IP pública fija	1 Gbps, IP pública fija
Respaldo	Copia diaria a almacenamiento externo	Copia diaria + copia semanal en otra región

Tabla 9.3 — Dimensionamiento del servidor (estimación; se ajusta con datos de uso real).

9.2.2 Dimensionamiento de datos

Con 7 sensores por galpón y una lectura cada 5 segundos se generan 120.960 lecturas por día. En la prueba de carga local, cada lectura ocupó 93 bytes incluyendo su índice (120.960 filas = 11 MB), de modo que el crecimiento estimado es de unos 11 MB por día, aproximadamente 4 GB por año por galpón sin compresión. La compresión nativa de TimescaleDB reduce ese volumen de forma significativa, por lo que los 40 GB del mínimo alcanzan para varios años del galpón actual. Si se agregaran galpones o se acortara el intervalo de lectura, el almacenamiento escala en proporción.

9.2.3 Dispositivos de campo y usuario

Elemento	Especificación
Nodo de sensores	Microcontrolador ESP32 · sensor de temperatura y humedad SHT31 o DHT22 · sensor de CO_2 MH-Z19 (calidad de aire) · caudalímetro de agua YF-S201 · fuente 5 V / 2 A · caja con protección contra polvo y humedad (IP65)
Red del galpón	Wi-Fi 2,4 GHz con cobertura en el punto de instalación; ancho de banda mínimo 1 Mbps de subida (cada lectura pesa menos de 100 bytes)
Celular / tablet	Android 8 o superior, o iOS 14 o superior; 2 GB de RAM; navegador Chrome o Safari actualizado
Computadora	Cualquier equipo con Chrome, Edge o Firefox actual; pantalla de 1366 × 768 o superior

Tabla 9.4 — Hardware de campo y de usuario.

9.3 Software de base y aplicación

Capa	Software y versión	Observación
Sistema operativo (servidor)	Ubuntu Server 24.04 LTS	Soporte extendido
Contenedores	Docker 24 o superior + Docker Compose	Despliegue reproducible
Proxy y TLS	Nginx 1.24 o superior · Let's Encrypt	Certificados renovables
Base de datos	PostgreSQL 15 o superior + TimescaleDB 2.x	Módulo IoT verificado en PostgreSQL 16.15
Broker MQTT	Eclipse Mosquitto 2.x	Pruebas automáticas con amqtt 0.12.1
Backend	Python 3.12 · Django 4.2 LTS · Django REST Framework · Channels 4 · Celery 5 · Redis 7	Según arquitectura de la Tesis (cap. 6)
Módulo de ingesta	Python 3 · paho-mqtt 2.1 · psycopg 3.3	Verificado (cap. 8)
Frontend	HTML/JS PWA (demo) → React 18 + Tailwind CSS 3 (versión de producción)	Service Worker para modo sin conexión
Hosting del frontend	Vercel	HTTPS y CDN incluidos
Integraciones	Don Gestión (iframe/REST) · AFIP-WSFE (Zeep SOAP)	AFIP requiere certificado digital del cliente
Navegadores soportados	Chrome / Edge / Firefox / Safari, últimas dos versiones principales	Probado en Chromium

Tabla 9.5 — Software de base y de aplicación.

9.4 Seguridad y auditoría

9.4.1 Autenticación y control de acceso

- Autenticación con JSON Web Tokens: token de acceso de 15 minutos y token de renovación de 7 días, almacenados en memoria y no en el almacenamiento local del navegador (Tesis, 13.2).
- Control de acceso por roles: Administrador, Supervisor Productivo, Operario de Galpón, Vendedor Comercial y Veterinario, cada uno con los módulos y permisos de la Tesis (13.1). Cada operario solo accede a los galpones que se le asignaron.
- Contraseñas almacenadas únicamente como hash (algoritmo PBKDF2 o Argon2 de Django), nunca en texto plano; límite de intentos fallidos de ingreso.

Importante: la demo funcional valida el usuario y la clave en el propio navegador, con dos cuentas de ejemplo, solo para mostrar los perfiles. Ese mecanismo no es seguro y no se utiliza en producción, donde la autenticación se resuelve en el servidor con JWT.

9.4.2 Protección de datos en tránsito y en reposo

- Todo el tráfico web viaja por HTTPS (TLS). Los sensores publican por MQTT sobre TLS (puerto 8883) con usuario y contraseña.
- El broker aplica listas de control de acceso por topic: cada nodo solo puede publicar en sus propios topics y el servicio de ingesta es el único suscriptor.
- La base de datos y Redis no se exponen a internet; solo son accesibles desde la red interna de los contenedores.
- Las credenciales (base de datos, MQTT, certificado AFIP) se guardan en variables de entorno fuera del repositorio.
- Validación de entrada: la ingesta descarta lecturas con formato inválido, valores fuera de rango físico y sensores no registrados (verificado en las pruebas IOT-20 a 27).

- Respaldos: copia diaria de la base (pg_dump) con retención de 30 días, almacenada fuera del servidor y cifrada.
- Los datos personales (usuarios y clientes) se tratan conforme a la Ley 25.326 de Protección de Datos Personales de la República Argentina y se limitan a los estrictamente necesarios.

9.4.3 Auditoría

Toda operación de escritura sobre las tablas auditadas queda registrada por triggers de PostgreSQL en la tabla auditoria, con usuario, fecha y hora, tabla, operación (INSERT, UPDATE o DELETE) y una copia en formato JSON del registro antes y después del cambio. Esto mantiene la trazabilidad incluso ante cambios hechos directamente sobre la base. El usuario de la aplicación se informa por transacción mediante SET LOCAL app.usuario.

El mecanismo fue implementado (backend/db/auditoria.sql) y probado: al marcar una alerta IoT como revisada con el usuario «supervisor1», quedó registrada la operación UPDATE con el valor anterior (revisada = falso) y el posterior (revisada = verdadero).

Usuario	Tabla	Operación	Antes	Después
supervisor1	alertas_iot	UPDATE	revisada = false	revisada = true

Tabla 9.6 — Registro de auditoría obtenido en la prueba.

Además del registro en base de datos, el servicio de ingesta deja en su log cada alerta generada y cada lectura descartada, lo que permite investigar fallas de sensores.

9.4.4 Riesgos de seguridad y medidas

Riesgo	Medida
Acceso no autorizado a la API	JWT de vida corta, roles y permisos por galpón, límite de intentos de ingreso
Suplantación de un sensor	Usuario y contraseña por nodo, TLS y ACL por topic en el broker
Pérdida de datos	Respaldo diario cifrado con retención de 30 días y prueba periódica de restauración
Datos falsos o corruptos en la base	Validación de formato y rango antes de guardar; descarte registrado en el log
Exposición de claves	Variables de entorno fuera del repositorio; rotación ante sospecha de filtración

Tabla 9.7 — Riesgos de seguridad y medidas de mitigación.